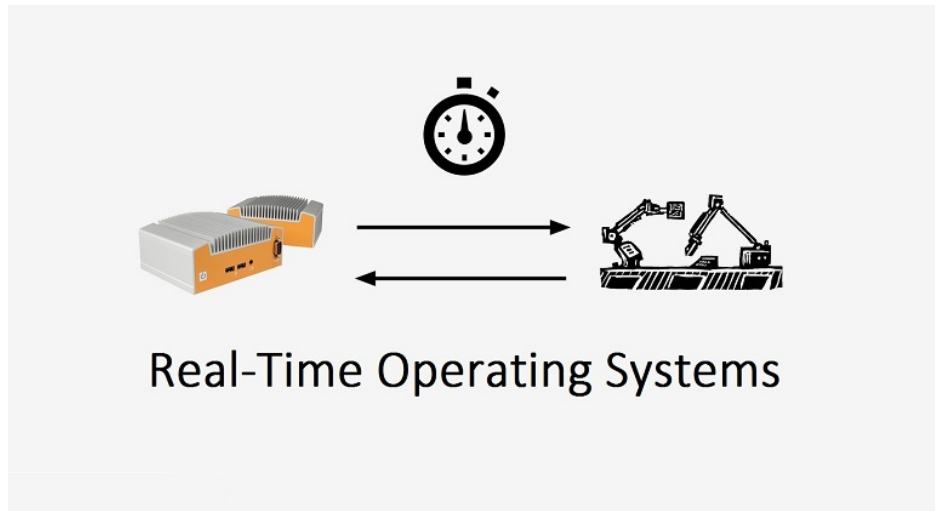




Real-time Operating Systems

LAB MODULE I - RT SERVICES ON LINUX
PART II - REAL-TIME APPLICATION - SIMULATION

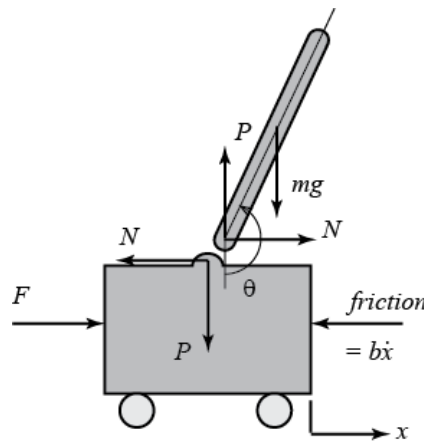
2026/2027

Paulo Pedreiras
DETI/UA/IT
pbrp@ua.pt

September 30, 2026

4 Part II: Development of a real-time application (simulated)

The objective of this second part of Lab Module I is to develop a real-time application in Linux that approaches a realistic scenario. To this end, it will be used the simulation of an Inverted Pendulum, which is a classical set-up used to learn instrumentation, control theory and real-time programming.



As the focus of this course unit is real-time and students lack background in Control Theory, it is given, as a starting point, a basic implementation of the simulator that despite performing correctly is monolithic, not follow the real-time model.

Students are then expected to evolve the provided software to obtain solutions that approach realistic implementations of a practical real-time system. More specifically, these exercises are designed to transition the monolithic architecture of `inverted_pendulum.c` into a distributed, multi-tasking real-time system using Linux inter-process communication (IPC) and scheduling features.

In reality, this kind of systems is often implemented in embedded and resource-constrained hardware. Moreover, the theoretic foundations taught in the first part of the semester address single-core computing platforms. As such, students should use the necessary mechanisms to force all application's threads to run in the same CPU, thus approaching a realistic situation for which they (will) have enough formal tools to analyze critically the system

behaviour.

In each step students should gather experimental data and compare it with the theoretical expectations.

4.1 Deconstructing the Control Loop - Real-Time model with TT+ET (1/2)

The current `control_loop_thread` executes at 200 *Hz* (5 *ms* period) and is monolithic, handling reading state, calculating the control law, and writing the actuation force in a single block.

- **Objective:** Split the control loop into three distinct threads to emulate a distributed hardware system: a Sensor thread, a Controller thread, and an Actuator thread.
- **Tasks:**
 - **Sensor Task:** Read `theta`, `theta_dot`, `x`, and `x_dot` from the plant. Send this data to the Controller task using a POSIX Message Queue.
 - **Controller Task:** Read the state from the message queue, apply the LQR state feedback law (using `K_theta`, `K_theta_dot`, `K_x`, `K_x_dot`), and send the computed `control_force` to the Actuator task via a second message queue.
 - **Actuator Task:** Read the force from the second message queue and update `g_state.force`.
- Assign a suitable real-time scheduling class and priorities to tasks, making the Sensor Task periodic and the other ones sporadic.
- Before proceeding, assure that the system works correctly.
- Measure the execution time of each task, and record the corresponding start and finish times. With this information draw a Gantt chart and reason about the schedulability of the system. Confirm your observations by matching them with the results obtained e.g. via `ftrace+kernelshark`.
- Generate load using tools like `stress-ng` and others and analyze the impact on the system performance

- Create a periodic load task, also with real-time priorities, and make its period, job execution time and priority change, taking note of the corresponding impact on system behaviour. Instead of taking just “guesses”, take some time to reason about the eventual impact of the load task in the system performance, before deciding about the load task job execution time and period ranges of interest.

Deliverables: `inverted_pendulum_TT_ET_MQ.c` and Gantt chart with annotations (tasks, priorities and message queues).

4.2 Deconstructing the Control Loop - Real-Time model with TT+ET (2/2)

Message queues are not the only way of allowing data sharing and synchronization among tasks. For example, it is possible to use shared memory (lowest overhead) combined with some sort of signaling. In Linux and other RTOSs, POSIX semaphores can be used to carry out such kind of task synchronization.

Note: In RT Linux programming semaphores are usually preferred to signals, because despite Linux providing a dedicated block of signals specifically for this purpose (ranging from SIGRTMIN to SIGRTMAX), its use is complex and restricts the software to a tiny list of async-signal-safe functions.

- Take the code developed in Section 4.1 and modify it so that the Sensor, Controller and Actuator communicate via shared memory and are synchronized via POSIX semaphores. Make sure the activation instants are properly controlled to assure the right execution sequence (Sensor → Controller → Actuator).
- Re-evaluate the priorities of the tasks and, if needed, modify them.
- Before proceeding, assure that the system works correctly.
- Measure the execution time of each task, and record the corresponding start and finish times. With this information draw a Gantt chart and confirm that the system is working as expected and reason about the its schedulability. Confirm your observations by matching them with the results obtained e.g. via `ftrace+kernelshark`.
- Generate load using tools like `stress-ng` and others and analyze the impact on the system performance

- Create a periodic load task, also with real-time priorities, and make its period, job execution time and priority change, taking note of the corresponding impact on system behaviour. Take some time to reason about the eventual impact of the load task in the system performance, and decide about the load and period ranges of interest to test.

Deliverables: `inverted_pendulum_TT_ET_SEM_SHM.c` and Gantt chart with annotations (tasks, priorities, sharemem and semaphores).

4.3 Deconstructing the Control Loop - Pure TT Real-Time model

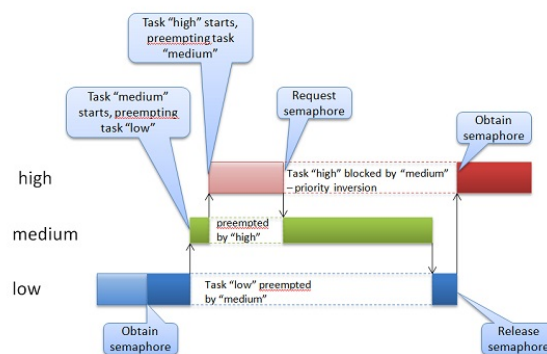
This kind of applications are often implemented according to a pure Time-Triggered architecture, in which all tasks are triggered by the system clock. This allows a precise control of task activations, permitting in many cases e.g. to distribute the load over time, reducing jitter.

- Take the code developed in Section 4.1 and modify it to make the Sensor, Controller and Actuator tasks all periodic. Make sure the activation instants are properly controlled to assure the right execution sequence (Sensor → Controller → Actuator)
- Re-evaluate the priorities of the task and modify them if necessary.
- Re-evaluate the use of message queues and eventually replace them for a more efficient technique.
- Before proceeding, assure that the system works correctly.
- Measure the execution time of each task, and record the corresponding start and finish times. With this information draw a Gantt chart and confirm that the system is working as expected and reason about the its schedulability. Confirm your observations by matching them with the results obtained e.g. via `ftrace+kernelshark`.
- Generate load using tools like `stress-ng` and others and analyze the impact on the system performance
- Create a periodic load task, also with real-time priorities, and make its period, job execution time and priority change, taking note of the corresponding impact on system behaviour. Take some time to reason about the eventual impact of the load task in the system performance, and decide about the load and period ranges of interest to test.

Deliverables: `inverted_pendulum_TT.c` and Gantt chart with annotations (tasks, priorities, IPC mechanism).

4.4 Verifying and addressing Priority Inversion (Synchronization)

Priority inversion may happen when a high priority (HP) task shares resources, protected by mutual exclusion, with a lower priority task (LP), and there are other tasks with intermediary that activate when the HP task is blocked by LP one.



The objective of this exercise is to observe the impact of priority inversion and use mechanism to prevent it.

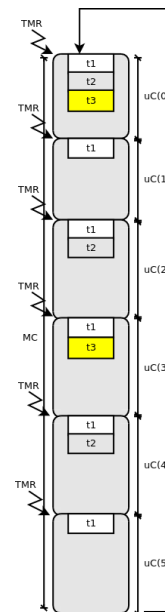
- Take the code developed in Section 4.2 and, if you have not done so, protect the shared memory areas with mutexes (simple ones, for now). At a first glance this may seem unnecessary, but in many systems this is an extremely important safeguard. In the general case, if the system gets overloaded it can be possible e.g. that the Actuator tasks does not finish before the next control cycle, and then the Controller task could write data to the shared memory area while the Actuator task is processing it, potentially corrupting data.
- Assign distinct and spaced priorities to the Sensor, Controller and Actuator tasks (e.g. 50, 40, 30, respectively). Make sure that the simulator task runs at an higher priority (e.g. 60).
- Before proceeding, assure that the system works correctly.

- Create a load task with e.g. priority 35. Make this task execute for one second every two seconds. Check the system behaviour.
- Execute the load task with other periods and job execution times and interpret the results.
- Take one of the scenarios in which the system under-performs, and change the mutex to PiP. Check the result.
- Measure the execution time of each task, and record the corresponding start and finish times. With this information draw a Gantt chart with and without PiP. Confirm that the system is working as expected and reason about its schedulability in both cases. Confirm your observations by matching them with the results obtained e.g. via `ftrace+kernelshark`.

Deliverables: `inverted_pendulum_PrioInv.c` and Gantt chart with annotations (tasks, priorities, sharemem and mutexes).

4.5 Static Table-Based Scheduling

While native Linux scheduling classes do not directly support Cyclic Table-Based Scheduling, it is fairly easy to implement such service at user level. The approach basically consists in creating a high-priority periodic task, whose period is the micro-cycle (let's call it the Tick task), that scans the scheduling table and triggers the tasks therein set. The Tick task should manage the table index, to move from tick to tick in each invocation (don't forget to handle the macro-cycle). Task can either be implemented as simple C functions or as threads. The latter approach is more complex but more powerful, as it allows to run tasks (threads) at specific priorities, e.g. at lower priority than the Tick task.



The objective of this exercise is to adapt the code in Section 4.3 to make it be executed by a user-level Cyclic Table-Based Scheduler. You can “hard-code” a solution or (preferably) implement the scheduler as a software module/library, with a well-defined API that can be used in arbitrary projects.

Such library should contain functions to Initialize the data structures, Add tasks, Build the table, Fire the system, etc.

- Implement the scheduler
- Adapt the tasks and the IPC mechanisms
- Before proceeding, assure that the system works correctly
- Measure the execution time of each task, and record the corresponding start and finish times. With this information draw a Gantt chart with and without PiP. Confirm that the system is working as expected and reason about the its schedulability in both cases. Confirm your observations by matching them with the results obtained e.g. via ftrace+kernelshark.

Deliverables: `inverted_pendulum_STB.c` + eventual library files and Gantt chart with annotations (tasks, priorities, sharemem).